Relatório de vulnerabilidades

Projeto: FIAP Tech Challenge Fase 1, backend da oficina.

Repositório: https://github.com/pedrohmirandadev/fiap-fase-1

Base analisada: branch `main`, commit `a8ae42c2277fcb7457e7ef905c77b33a093f68ad`, com revisão posterior das alterações locais descritas abaixo.

Data da análise: 30/10/2026.

Método: revisão estática de código e configuração, além de auditoria de dependências.

O hash identifica o commit-base, anterior a este relatório e às correções implementadas na branch `feat/fiap-fase1-pdf-requirements` (commit `5afff0`). Até o merge dessa branch, a versão `main` continua com os achados SEC-001 e SEC-002. A revisão não alterou o lockfile.

Resumo

A análise do commit-base encontrou três pontos a corrigir antes de expor o sistema a redes não confiáveis. Na branch de entrega, a aplicação rejeita uma chave JWT ausente ou curta e as portas do Compose ficam limitadas a `127.0.0.1`. Para facilitar a avaliação do professor, o Compose oferece credenciais e dados fictícios conhecidos, restritos à execução local. O login ainda não limita tentativas.

A auditoria do lockfile não encontrou vulnerabilidades conhecidas nas dependências consultadas. O código de acompanhamento tem aleatoriedade forte. Quem o possuir pode consultar os dados da OS e decidir sobre o orçamento enquanto o código continuar válido.

ID	Severidade	Achado	Situação na branch
SEC-001	Alta no commit-base	Chave JWT previsível permite falsificar autenticação administrativa	A aplicação rejeita segredo ausente/curto; o Compose usa uma chave de demonstração conhecida apenas para a avaliação local
SEC-002	Média	Compose publica serviços e contém credenciais padrão de demonstração	Mitigado para avaliação local nesta branch; ainda não está em `main`
SEC-003	Média	Login sem limitação de tentativas facilita ataques de senha online	Aberto
Risco residual	Baixa	Código de acompanhamento funciona como credencial sem expiração	Aceito no fluxo atual; rever expiração e revogação

As severidades são qualitativas e levam em conta o impacto e as condições de exposição descritas abaixo. Não foi calculado CVSS.

Achados

SEC-001 - Chave JWT padrão previsível

Severidade: Alta

Status: A aplicação exige configuração (correção nesta branch, ainda não em `main`); o Compose injeta uma chave pública de demonstração, adequada somente ao uso local.

Evidência no commit-base: `src/presentation/app.ts` escolhia uma chave fixa quando `JWT_SECRET` não estava definida, e o Compose também fornecia um padrão.

Verificação na branch: `createApp` agora encerra a criação da API se a chave estiver ausente ou tiver menos de 32 caracteres. O Compose injeta uma chave de demonstração conhecida para a avaliação local. Um teste de controller cobre as duas rejeições.

No commit-base, uma pessoa que soubesse a chave padrão poderia criar um JWT e acessar rotas administrativas de uma instância exposta que a mantivesse. A aplicação deixou de escolher um fallback por conta própria e exige uma chave configurada com pelo menos 32 caracteres antes de inicializar. Para a avaliação, o Compose fornece explicitamente um valor conhecido; como a API só publica a porta em `127.0.0.1`, esse modo atende ao uso local, mas não deve ser exposto a outras máquinas.

Operação: o Compose fornece uma chave conhecida apenas para a demonstração local; ela não é um segredo e não deve ser reaproveitada fora da avaliação. A aplicação continua exigindo uma chave de pelo menos 32 caracteres. A OWASP JWT Cheat Sheet explica que qualquer validador que compartilhe uma chave MAC também pode emitir tokens.

SEC-002 - Credenciais de demonstração e portas publicadas pelo Compose

Severidade: Média

Status: Mitigado para a avaliação local nesta branch; ainda não está em `main`.

Evidência no commit-base: `docker-compose.yml` publicava portas em todas as interfaces e definia valores de demonstração.

Verificação na branch: os serviços publicados se vinculam a `127.0.0.1`. Para o professor executar o sistema sem configuração prévia, o Compose usa valores fictícios conhecidos para o banco, JWT e conta administrativa. O `.env.example` registra os mesmos valores para execução da API diretamente no host. O seed cria apenas dados de demonstração.

No commit-base, outras máquinas com acesso ao host poderiam alcançar a API, o PostgreSQL e, quando ativado, o SonarQube. A branch restringe as portas publicadas ao loopback; ao executar Bun diretamente no host, a API também escuta em `127.0.0.1` por padrão. No container, o processo escuta na rede interna e o Compose publica a porta somente no loopback do computador. Os valores previsíveis permanecem intencionais para a avaliação local: não são segredos e qualquer processo executado na mesma máquina pode usá-los. O perfil `test` cria um PostgreSQL separado sem porta publicada no host e executa os testes sem seed. Consulte a documentação do Docker sobre publicação de portas.

Limite de uso: esta configuração existe para professor e grupo iniciarem e avaliarem o projeto localmente. Não exponha as portas, não use os valores conhecidos em computadores compartilhados e não trate a configuração de demonstração como proteção para um serviço publicado. O perfil SonarQube é opcional e também se limita ao loopback.

SEC-003 - Endpoint de login sem limitação de tentativas

Severidade: Média

Status: Aberto

Evidência: `src/presentation/app.ts` registra `POST /auth/login` sem middleware de throttling; `src/application/use-cases/authenticate-admin.ts` autentica as credenciais sem registrar ou limitar falhas.

O código não limita tentativas por conta, origem ou período. Se a API estiver exposta, ferramentas automatizadas podem testar senhas ou reutilizar credenciais vazadas sem bloqueio da aplicação. O bcrypt torna cada tentativa mais custosa, mas não limita quantas podem ser feitas.

Correção recomendada: limite tentativas por conta e por origem, aplique atrasos progressivos e registre falhas para monitoramento. Se houver bloqueio temporário, configure-o para não facilitar ataques de negação de serviço contra uma conta. A OWASP Authentication Cheat Sheet descreve controles de login throttling.

Risco residual: acompanhamento público da OS

O sistema gera o código de acompanhamento a partir de 24 bytes aleatórios criptograficamente seguros (`src/domain/shared/value-objects.ts`), o que dificulta adivinhá-lo. Quem tem o código pode consultar a placa, a reclamação, os itens e os valores (`src/infrastructure/queries/work-order-queries.ts`), além de aprovar ou rejeitar o orçamento (`src/presentation/routes/work-order-routes.ts`). O código não expira e não há mecanismo de revogação.

O acesso por código atende ao fluxo de acompanhamento previsto. A análise não encontrou falha na aleatoriedade nem bypass de autenticação. Trate o código como uma credencial: envie-o apenas ao cliente, evite colocá-lo em logs ou em links que possam ser compartilhados com terceiros e use HTTPS. Considere também expiração, revogação e limitar os dados da resposta pública.

O projeto não define uma chave de rastreamento separada; o código aleatório não expira nem pode ser revogado individualmente.

Auditoria de dependências

- **Ferramenta:** Bun 1.4.4 (`bun audit`), versão oficial `38afee6c8`.
- **Escopo:** versões no `bun.lock`, incluindo dependências diretas e transitivas reconhecidas pela ferramenta.
- **Resultado em 30/10/2026:** `No vulnerabilities found (checked 30 packages)`; código de saída 0.
- **Evidência detalhada:** `dependency-audit.md`.

O comando consulta avisos de dependências no registry/advisory endpoint e não altera o lockfile, conforme a documentação oficial do Bun. O resultado cobre os avisos conhecidos para as versões consultadas nessa data. Ele não avalia o código nem a infraestrutura.

Escopo e limitações

Esta revisão estática cobriu autenticação JWT, login, rotas públicas de acompanhamento, consultas SQL, validação das rotas, arquivos Docker/Compose e dependências do lockfile. As consultas SQL examinadas usam parâmetros; não encontramos injeção SQL nesse escopo.

Não fizemos testes de penetração ou varreduras dinâmicas. A análise também não cobre CVEs das imagens Docker nem uma busca completa por segredos no histórico Git. O relatório descreve a árvore baseada no commit informado e não garante que o sistema esteja livre de vulnerabilidades. Repita a análise após mudanças no código, nas dependências ou na configuração.

Validações das alterações locais

Em 30/10/2026, `bun run test:coverage` concluiu com **209 testes aprovados, 0 falhas e 0 ignorados**. Foram medidos 105 arquivos, com 99,93% de cobertura global de linhas e 100% de funções; nenhum arquivo ficou abaixo do limite de 90%. A execução e os números estão registrados na evidência de cobertura. Esses testes verificam comportamento e cobertura, mas não substituem uma varredura dinâmica ou um teste de penetração.

Prioridade de correção

1. Publicar as correções locais de JWT e Compose após revisão.
2. Adicionar throttling e monitoramento ao login.
3. Definir prazo/revogação para códigos públicos e reduzir sua exposição em logs, links e respostas.